이 파트에서는 **데이터베이스** 기초와 **RAG(Retrieval-Augmented Generation)** 기술을 학습합니다.

SQL 기본 문법부터 시작하여, LLM의 한계(할루시네이션, 지식 컷오프)를 보완하는 RAG 아키텍처를 이해하고, 실제로 사내 문서 기반 QA 시스템을 구축합니다.

학습 목표

- SQL 기본 문법(CRUD, JOIN, 집계함수)을 사용할 수 있습니다.
- Python에서 MySQL 데이터베이스에 연결하여 데이터를 처리할 수 있습니다.
- RAG의 전체 파이프라인(청킹 → 임베딩 → 벡터 저장 → 검색 → 생성)을 이해합니다.
- ChromaDB와 GPT를 연동하여 문서 기반 QA 시스템을 구축할 수 있습니다.

이 파트의 구성

챕터	제목	핵심 내용
Chapter 19	데이터베이스와 SQL 기초	MySQL, SQL 문법, Python DB 연결
Chapter 20	RAG (Retrieval-Augmented Generation)	임베딩, 벡터 DB, 청킹, QA 시스템

Chapter 19. 데이터베이스와 SQL 기초

현대 소프트웨어 개발에서 데이터를 체계적으로 저장하고 관리하는 능력은 필수입니다. 파일로 데이터를 저장하는 방식은 간단하지만, 데이터가 많아지고 복잡해질수록 한계가 드러납니다. 이때 필요한 것이 바로 **데이터베이스(Database)**입니다.

이 챕터에서는 데이터베이스의 기본 개념부터 시작하여 가장 널리 사용되는 **관계형 데이터베이스**의 구조를 이해하고, **SQL(Structured Query Language)**로 데이터를 다루는 방법을 학습합니다. 나아가 Python에서 데이터베이스를 연결하고 실제로 데이터를 조작하는 실습까지 진행합니다.

이 챕터에서 배울 내용:

- 데이터베이스의 개념과 관계형 DB vs NoSQL의 차이
- 테이블, 행, 열, 키 등 관계형 DB의 핵심 구조
- MySQL 설치 및 기본 설정 방법
- SELECT, INSERT, UPDATE, DELETE 기본 SQL 문법
- WHERE, ORDER BY, LIMIT을 활용한 데이터 조회
- JOIN과 집계 함수를 활용한 고급 쿼리
- Python에서 pymysql과 SQLAlchemy로 DB 연결 및 CRUD 실습

19.1 데이터베이스의 개념과 종류

데이터베이스(Database, DB)란 데이터를 구조화하여 저장하고, 효율적으로 검색·수정·삭제할 수 있도록 관리하는 시스템입니다. 데이터베이스를 관리하는 소프트웨어를 **DBMS(Database Management System)**라고 합니다.

데이터베이스는 크게 **관계형 데이터베이스(RDBMS)**와 **NoSQL 데이터베이스**로 나뉩니다.

관계형 데이터베이스 (RDBMS)

관계형 데이터베이스(Relational Database Management System)는 데이터를 행(row)과 열(column)로 이루어진 테이블 형태로 저장합니다. 테이블 간의 관계를 정의하여 데이터 중복을 줄이고 일관성을 유지합니다. SQL을 사용하여 데이터를 조작합니다.

대표적인 RDBMS로는 MySQL, PostgreSQL, Oracle, SQLite, MariaDB 등이 있습니다.

NoSQL 데이터베이스

NoSQL(Not Only SQL)은 관계형 모델을 따르지 않는 다양한 형태의 데이터베이스를 통칭합니다. 대용량 비정형 데이터 처리에 적합하며 수평 확장이 용이합니다.

구분	관계형 DB (RDBMS)	NoSQL
데이터 구조	테이블 (행/열)	문서, 키-값, 그래프 등
스키마	고정 스키마	유연한 스키마
쿼리 언어	SQL	각 DB별 고유 문법
확장성	수직 확장	수평 확장
일관성	강한 일관성 (ACID)	최종 일관성 (BASE)
대표 제품	MySQL, PostgreSQL	MongoDB, Redis, Cassandra
적합한 용도	정형 데이터, 복잡한 관계	대용량, 비정형 데이터

이 챕터에서는 가장 널리 사용되는 관계형 데이터베이스인 **MySQL**을 중심으로 학습합니다.

> 💡 ***어떤 DB를 선택해야 할까?*** *트랜잭션이 중요하고 데이터 관계가 복잡하다면 RDBMS, 빠른 읽기/쓰기와 유연한 구조가 필요하다면 NoSQL을 고려하세요. 실제 프로젝트에서는 두 가지를 혼합하여 사용하기도 합니다.*

19.2 관계형 데이터베이스 구조

관계형 데이터베이스의 핵심 구성 요소를 이해하면 SQL을 훨씬 쉽게 배울 수 있습니다.

테이블 (Table)

테이블(Table)은 관계형 데이터베이스의 기본 저장 단위입니다. 스프레드시트의 시트와 유사하게 행과 열로 데이터를 저장합니다. 하나의 데이터베이스는 여러 개의 테이블로 구성됩니다.

예를 들어, 쇼핑몰 데이터베이스에는 `users`(회원), `products`(상품), `orders`(주문) 등의 테이블이 있을 수 있습니다.

행과 열 (Row & Column)

행(Row)은 하나의 데이터 레코드를 의미합니다. `users` 테이블에서 한 행은 한 명의 회원 정보를 나타냅니다. **열(Column)**은 데이터의 속성(필드)을 의미합니다. 이름, 이메일, 가입일 등이 열에 해당합니다.

id	name	email	created_at
1	홍길동	hong@example.com	2024-01-15
2	김철수	kim@example.com	2024-02-20
3	이영희	lee@example.com	2024-03-10

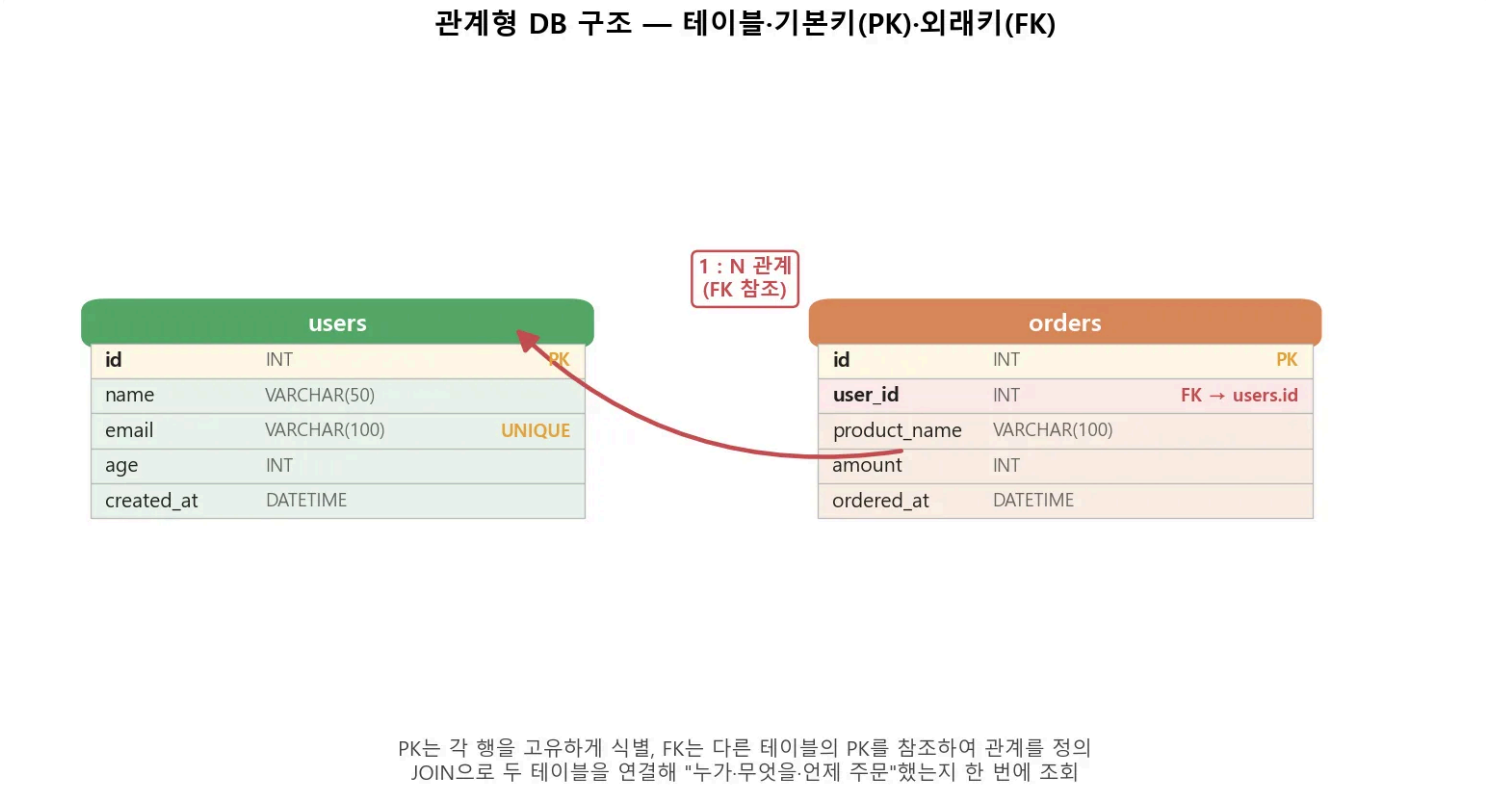
키 (Key)

키(Key)는 테이블에서 행을 식별하거나 테이블 간의 관계를 정의하는 데 사용됩니다.

기본 키(Primary Key, PK)는 테이블에서 각 행을 고유하게 식별하는 열입니다. 중복될 수 없고 NULL 값을 가질 수 없습니다. 위 예시의 `id` 열이 기본 키입니다.

외래 키(Foreign Key, FK)는 다른 테이블의 기본 키를 참조하는 열입니다. 테이블 간의 관계를 정의합니다. `orders` 테이블의 `user_id`가 `users` 테이블의 `id`를 참조하는 외래 키가 됩니다.

> 📌 ***스키마(Schema)란?*** *스키마는 데이터베이스의 구조를 정의하는 설계도입니다. 테이블 이름, 각 열의 이름과 데이터 타입, 제약 조건, 테이블 간의 관계 등을 포함합니다.*



19.3 MySQL 설치 및 기본 설정

MySQL을 설치하고 기본 환경을 설정하는 방법을 알아봅니다.

MySQL 설치

운영체제별 설치 방법입니다.

Windows: MySQL 공식 사이트에서 MySQL Installer를 다운로드하여 설치합니다. MySQL Server와 MySQL Workbench를 함께 설치하면 편리합니다.

macOS: Homebrew를 사용하는 것이 간편합니다.

```
brew install mysql
brew services start mysql
```

Ubuntu/Linux: apt 패키지 관리자를 사용합니다.

```
sudo apt update
sudo apt install mysql-server
sudo systemctl start mysql
sudo mysql_secure_installation
```

MySQL 접속 및 기본 명령

설치 후 터미널에서 MySQL에 접속합니다.

```
mysql -u root -p
```

MySQL에 접속한 후 기본적인 관리 명령어를 익혀 둡니다.

```
-- 데이터베이스 목록 확인
SHOW DATABASES;

-- 데이터베이스 생성
CREATE DATABASE myshop CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- 데이터베이스 선택
USE myshop;

-- 테이블 목록 확인
SHOW TABLES;

-- MySQL 종료
EXIT;
```

테이블 생성

실습에 사용할 `users` 테이블을 생성합니다.

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    age INT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

주요 데이터 타입은 다음과 같습니다.

데이터 타입	설명	예시
`INT`	정수	나이, ID
`VARCHAR(n)`	가변 길이 문자열 (최대 n자)	이름, 이메일
`TEXT`	긴 문자열	본문, 댓글
`DATETIME`	날짜와 시간	생성일, 수정일
`DECIMAL(p,s)`	소수점 포함 실수	가격, 금액
`BOOLEAN`	참/거짓	활성화 여부

> ⚠️ ***한글 인코딩 주의*** *MySQL에서 한글을 올바르게 저장하려면 데이터베이스와 테이블 생성 시 `utf8mb4` 인코딩을 지정해야 합니다. `utf8mb4`는 이모지를 포함한 모든 유니코드 문자를 지원합니다.*

19.4 SQL 기초 문법 - SELECT, INSERT, UPDATE, DELETE

SQL의 데이터 조작 언어(DML)의 핵심 4가지 명령어를 **CRUD**라고도 합니다. **C**reate(INSERT), **R**ead(SELECT), **U**pdate(UPDATE), **D**elete(DELETE)를 의미합니다.

실습을 위해 먼저 샘플 데이터를 삽입합니다.

```
INSERT INTO users (name, email, age) VALUES
('홍길동', 'hong@example.com', 28),
('김철수', 'kim@example.com', 35),
('이영희', 'lee@example.com', 24),
('박민준', 'park@example.com', 42),
('최수진', 'choi@example.com', 31);
```

SELECT - 데이터 조회

`SELECT` 는 테이블에서 데이터를 조회하는 명령어입니다.

```
-- 모든 열 조회
SELECT * FROM users;

-- 특정 열만 조회
SELECT name, email FROM users;

-- 열에 별칭(Alias) 지정
SELECT name AS 이름, email AS 이메일 FROM users;
```

INSERT - 데이터 삽입

`INSERT INTO` 는 테이블에 새로운 행을 추가합니다.

```
-- 단일 행 삽입
INSERT INTO users (name, email, age)
VALUES ('정다은', 'jung@example.com', 27);

-- AUTO_INCREMENT 열(id)은 생략해도 자동으로 값이 부여됩니다
```

UPDATE - 데이터 수정

`UPDATE` 는 기존 데이터를 수정합니다.

```
-- 특정 사용자의 나이 수정
UPDATE users
SET age = 29
WHERE name = '홍길동';

-- 여러 열 동시 수정
UPDATE users
SET age = 36, email = 'kim_new@example.com'
WHERE id = 2;
```

> ⚠️ ***WHERE 절 없이 UPDATE 금지*** *`WHERE` 절 없이 `UPDATE`를 실행하면 테이블의 모든 행이 수정됩니다. 반드시 조건을 지정하는 습관을 들이세요.*

DELETE - 데이터 삭제

`DELETE`는 테이블에서 행을 삭제합니다.

```
-- 특정 행 삭제
DELETE FROM users WHERE id = 6;

-- 조건에 맞는 여러 행 삭제
DELETE FROM users WHERE age > 40;
```

> ⚠️ ***WHERE 절 없이 DELETE 금지*** *`WHERE` 절 없이 `DELETE FROM users;`를 실행하면 테이블의 모든 데이터가 삭제됩니다. 삭제 전에 `SELECT`로 먼저 대상 데이터를 확인하는 습관을 들이세요.*

19.5 조건절(WHERE), 정렬(ORDER BY), 제한(LIMIT)

실제 데이터 조회에서는 원하는 데이터만 필터링하고 정렬하는 기능이 핵심입니다.

WHERE - 조건 필터링

`WHERE` 절은 조회할 데이터의 조건을 지정합니다.

```sql
-- 나이가 30 이상인 사용자 조회
SELECT * FROM users WHERE age >= 30;

-- 이름이 '홍길동'인 사용자 조회
SELECT * FROM users WHERE name = '홍길동';

-- AND / OR 연산자 사용
SELECT * FROM users WHERE age >= 25 AND age <= 35;
SELECT * FROM users WHERE name = '홍길동' OR name = '김철수';

-- BETWEEN: 범위 조건
SELECT * FROM users WHERE age BETWEEN 25 AND 35;

-- IN: 여러 값 중 하나와 일치
SELECT * FROM users WHERE age IN (24, 28, 31);

-- LIKE: 패턴 매칭 (% = 임의의 문자열, _ = 임의의 한 문자)
SELECT * FROM users WHERE email LIKE '%@example.com';
SELECT * FROM users WHERE name LIKE '김%';

-- IS NULL / IS NOT NULL
SELECT * FROM users WHERE age IS NULL;
```

ORDER BY - 정렬

`ORDER BY`는 결과를 특정 열 기준으로 정렬합니다.

```sql
-- 나이 오름차순 정렬 (기본값: ASC)
SELECT * FROM users ORDER BY age ASC;

-- 나이 내림차순 정렬
SELECT * FROM users ORDER BY age DESC;

-- 여러 열 기준 정렬 (이름 오름차순, 나이 내림차순)
SELECT * FROM users ORDER BY name ASC, age DESC;
```

LIMIT - 결과 수 제한

`LIMIT`은 반환할 행의 수를 제한합니다. 페이지네이션 구현에도 활용됩니다.

```
-- 상위 3개 행만 조회
SELECT * FROM users LIMIT 3;

-- 나이 높은 순으로 상위 2명 조회
SELECT * FROM users ORDER BY age DESC LIMIT 2;

-- OFFSET: 시작 위치 지정 (페이지네이션)
-- 4번째 행부터 3개 조회 (2페이지, 페이지당 3개)
SELECT * FROM users LIMIT 3 OFFSET 3;
```

> 💡 ***쿼리 실행 순서*** *SQL 쿼리는 작성 순서와 실행 순서가 다릅니다.* `FROM` → `WHERE` → `SELECT` → `ORDER BY` → `LIMIT` *순서로 처리됩니다. 이를 이해하면 복잡한 쿼리 작성 시 오류를 예방할 수 있습니다.*

19.6 조인(JOIN)과 집계 함수

실제 데이터베이스에서는 여러 테이블을 연결하거나 데이터를 요약할 일이 많습니다.

JOIN - 테이블 연결

실습을 위해 `orders` 테이블을 생성하고 데이터를 추가합니다.

```
CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    amount INT NOT NULL,
    ordered_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

INSERT INTO orders (user_id, product_name, amount) VALUES
(1, '노트북', 1200000),
(1, '마우스', 30000),
(2, '키보드', 80000),
(3, '모니터', 350000),
(2, '웹캠', 60000);
```

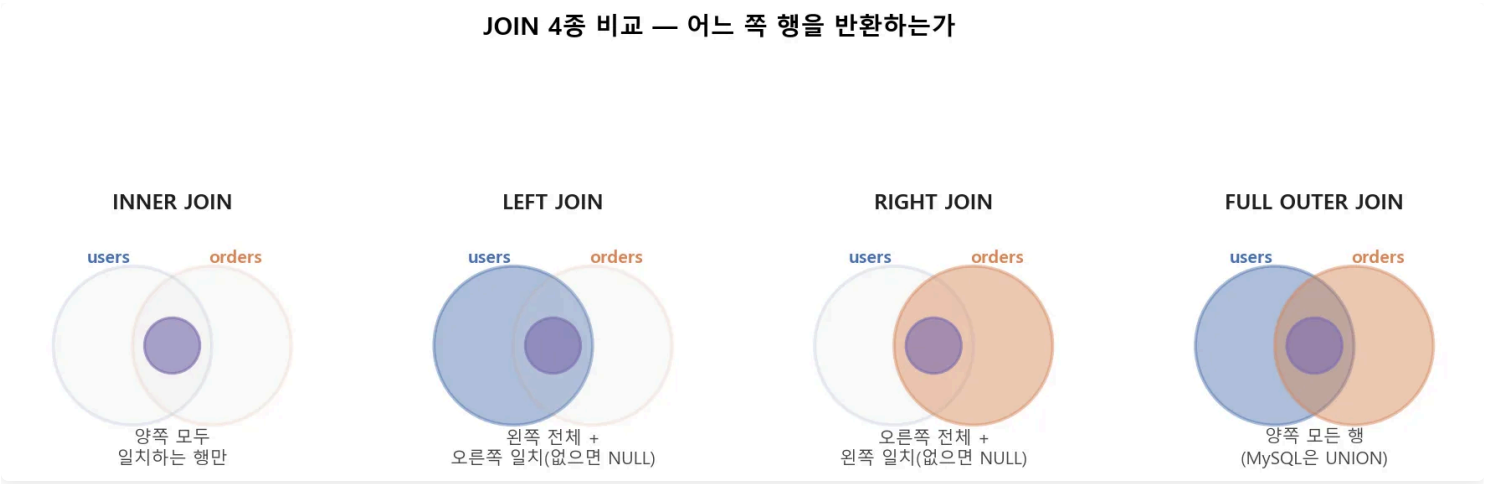
INNER JOIN은 두 테이블에서 조건이 일치하는 행만 반환합니다.

```
-- 주문 정보와 주문자 이름을 함께 조회
SELECT
    orders.id AS 주문번호,
    users.name AS 주문자,
    orders.product_name AS 상품명,
    orders.amount AS 금액
FROM orders
INNER JOIN users ON orders.user_id = users.id;
```

LEFT JOIN은 왼쪽 테이블의 모든 행을 반환하고, 오른쪽 테이블에 일치하는 행이 없으면 NULL을 반환합니다.

```
-- 주문이 없는 사용자도 포함하여 조회
SELECT
    users.name AS 이름,
    orders.product_name AS 상품명
FROM users
LEFT JOIN orders ON users.id = orders.user_id;
```

JOIN 종류	설명
`INNER JOIN`	양쪽 테이블에 모두 일치하는 행만 반환
`LEFT JOIN`	왼쪽 테이블 전체 + 오른쪽 일치 행 (없으면 NULL)
`RIGHT JOIN`	오른쪽 테이블 전체 + 왼쪽 일치 행 (없으면 NULL)
`FULL OUTER JOIN`	양쪽 테이블 모든 행 (MySQL은 UNION으로 구현)



집계 함수와 GROUP BY

집계 함수(Aggregate Function)는 여러 행의 데이터를 하나의 값으로 요약합니다.

```
-- COUNT: 행의 수
SELECT COUNT(*) AS 전체회원수 FROM users;
SELECT COUNT(*) AS 주문건수 FROM orders;

-- SUM: 합계
SELECT SUM(amount) AS 총매출 FROM orders;

-- AVG: 평균
SELECT AVG(amount) AS 평균주문금액 FROM orders;

-- MAX / MIN: 최댓값 / 최솟값
SELECT MAX(amount) AS 최고금액, MIN(amount) AS 최저금액 FROM orders;
```

`GROUP BY`는 특정 열의 값을 기준으로 행을 그룹화하고, 각 그룹에 집계 함수를 적용합니다.

```sql
-- 사용자별 주문 건수와 총 주문 금액
SELECT
    users.name AS 이름,
    COUNT(orders.id) AS 주문건수,
    SUM(orders.amount) AS 총주문금액
FROM users
LEFT JOIN orders ON users.id = orders.user_id
GROUP BY users.id, users.name;

-- HAVING: 그룹화 후 조건 필터링 (WHERE는 그룹화 전)
-- 총 주문금액이 100,000원 이상인 사용자만 조회
SELECT
    users.name AS 이름,
    SUM(orders.amount) AS 총주문금액
FROM users
INNER JOIN orders ON users.id = orders.user_id
GROUP BY users.id, users.name
HAVING SUM(orders.amount) ≥ 100000;
```

> 💡 ***WHERE vs HAVING*** *`WHERE` 는 그룹화(`GROUP BY`) 이전에 개별 행을 필터링하고, `HAVING` 은 그룹화 이후 그룹 전체를 필터링합니다. 집계 함수 조건은 반드시 `HAVING` 을 사용해야 합니다.*

19.7 Python에서 DB 연결하기

Python에서 MySQL에 연결하는 방법은 크게 두 가지입니다. 저수준 드라이버인 **pymysql**과 ORM(Object Relational Mapping) 라이브러리인 **SQLAlchemy**입니다.

pymysql 사용하기

`pymysql` 은 Python에서 MySQL에 직접 연결하여 SQL을 실행할 수 있는 라이브러리입니다.

```bash
pip install pymysql
```

기본 연결 및 쿼리 실행 방법입니다.

```
import pymysql

# 데이터베이스 연결
conn = pymysql.connect(
    host='localhost',
    user='root',
    password='your_password',
    database='myshop',
    charset='utf8mb4'
)

# 커서 생성
cursor = conn.cursor()

# SELECT 쿼리 실행
cursor.execute("SELECT * FROM users")
rows = cursor.fetchall()  # 모든 결과 가져오기

for row in rows:
    print(row)

# 연결 종료
cursor.close()
conn.close()
```

`fetchall()` 외에도 `fetchone()`(한 행), `fetchmany(n)`(n개 행)을 사용할 수 있습니다.

SQLAlchemy 사용하기

SQLAlchemy는 Python의 대표적인 ORM 라이브러리입니다. SQL을 직접 작성하지 않고 Python 객체로 데이터베이스를 다룰 수 있습니다. Django, Flask, FastAPI 등 많은 웹 프레임워크와 함께 사용됩니다.

```
pip install sqlalchemy pymysql
```

SQLAlchemy를 사용한 연결과 기본 쿼리 방법입니다.

```
from sqlalchemy import create_engine, text

# 데이터베이스 URL 형식: dialect+driver://user:password@host/dbname
DATABASE_URL = "mysql+pymysql://root:your_password@localhost/myshop?charset=utf8mb4"

# 엔진 생성
engine = create_engine(DATABASE_URL, echo=True)

# SQL 직접 실행 (Core 방식)
with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM users"))
    for row in result:
        print(row)
```

구분	pymysql	SQLAlchemy
방식	저수준 드라이버	ORM / SQL 표현식
SQL 작성	직접 SQL 문자열 작성	Python 객체 또는 SQL
학습 난이도	낮음	보통
사용 사례	간단한 스크립트	웹 애플리케이션, 대형 프로젝트
DB 교체 용이성	어려움	쉬움 (DB 추상화)

> 💡 ***어떤 것을 선택해야 할까?*** *간단한 스크립트나 데이터 분석 작업에는 `pymysql` 이 편리합니다. 웹 서비스나 규모 있는 애플리케이션을 개발한다면 `SQLAlchemy` 를 배워두는 것을 권장합니다.*

[실습] Python으로 데이터 CRUD 실습

지금까지 배운 내용을 Python 코드로 통합하는 CRUD 실습을 진행합니다. `pymysql` 을 사용하여 회원 관리 기능을 구현합니다.

실습 준비

먼저 필요한 라이브러리를 설치하고 데이터베이스가 준비되어 있는지 확인합니다.

```
pip install pymysql
```

DB 연결 모듈 작성

재사용 가능한 DB 연결 함수를 작성합니다.

```
# db_config.py
import pymysql

def get_connection():
    """MySQL 데이터베이스 연결을 반환합니다."""
    return pymysql.connect(
        host='localhost',
        user='root',
        password='your_password',  # 실제 비밀번호로 변경
        database='myshop',
        charset='utf8mb4',
        cursorclass=pymysql.cursors.DictCursor  # 결과를 딕셔너리로 반환
    )
```

`DictCursor`를 사용하면 결과를 딕셔너리 형태로 받아 `row['name']`처럼 열 이름으로 접근할 수 있습니다.

Create - 데이터 삽입

```
# crud_example.py
from db_config import get_connection

def create_user(name: str, email: str, age: int) → int:
    """새 사용자를 데이터베이스에 삽입하고 생성된 ID를 반환합니다."""
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            sql = "INSERT INTO users (name, email, age) VALUES (%s, %s, %s)"
            cursor.execute(sql, (name, email, age))
        conn.commit()  # 변경사항 저장
        return cursor.lastrowid  # 새로 생성된 행의 ID
    except Exception as e:
        conn.rollback()  # 오류 시 롤백
        print(f"오류 발생: {e}")
        raise
    finally:
        conn.close()

# 사용 예시
new_id = create_user('정다은', 'jung@example.com', 27)
print(f"새 사용자 생성 완료. ID: {new_id}")
```

Read - 데이터 조회

```
def get_all_users() → list:
    """모든 사용자를 조회합니다."""
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            cursor.execute("SELECT * FROM users ORDER BY id")
            return cursor.fetchall()
    finally:
        conn.close()

def get_user_by_id(user_id: int) → dict:
    """ID로 특정 사용자를 조회합니다."""
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            sql = "SELECT * FROM users WHERE id = %s"
            cursor.execute(sql, (user_id,))
            return cursor.fetchone()  # 단일 행 반환 (없으면 None)
    finally:
        conn.close()

# 사용 예시
users = get_all_users()
for user in users:
    print(f"[{user['id']}] {user['name']} ({user['email']}) - {user['age']}세")

user = get_user_by_id(1)
if user:
    print(f"\n조회된 사용자: {user['name']}")
```

Update - 데이터 수정

```
def update_user_age(user_id: int, new_age: int) → bool:
    """사용자의 나이를 수정합니다. 성공 여부를 반환합니다."""
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            sql = "UPDATE users SET age = %s WHERE id = %s"
            cursor.execute(sql, (new_age, user_id))
            affected_rows = cursor.rowcount  # 변경된 행 수
        conn.commit()
        return affected_rows > 0
    except Exception as e:
        conn.rollback()
        print(f"오류 발생: {e}")
        raise
    finally:
        conn.close()

# 사용 예시
success = update_user_age(1, 29)
print(f"수정 {'성공' if success else '실패'} (대상 없음)")
```

Delete - 데이터 삭제

```
def delete_user(user_id: int) → bool:
    """사용자를 삭제합니다. 성공 여부를 반환합니다."""
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            sql = "DELETE FROM users WHERE id = %s"
            cursor.execute(sql, (user_id,))
            affected_rows = cursor.rowcount
        conn.commit()
        return affected_rows > 0
    except Exception as e:
        conn.rollback()
        print(f"오류 발생: {e}")
        raise
    finally:
        conn.close()

# 사용 예시
success = delete_user(6)
print(f"삭제 {'성공' if success else '실패 (대상 없음)'}")
```

전체 CRUD 통합 실행

```
# main.py
from crud_example import create_user, get_all_users, get_user_by_id
from crud_example import update_user_age, delete_user

print("═══ 사용자 관리 CRUD 실습 ===\n")

# 1. 사용자 생성 (Create)
print("1. 새 사용자 생성")
new_id = create_user('테스트유저', 'test@example.com', 25)
print(f"   생성된 ID: {new_id}\n")

# 2. 전체 조회 (Read)
print("2. 전체 사용자 조회")
users = get_all_users()
for user in users:
    print(f"   [{user['id']}] {user['name']} - {user['age']}세")
print()

# 3. 단일 조회 (Read)
print(f"3. ID={new_id} 사용자 조회")
user = get_user_by_id(new_id)
print(f"   이름: {user['name']}, 이메일: {user['email']}\n")

# 4. 수정 (Update)
print(f"4. ID={new_id} 사용자 나이 수정 (25 → 26)")
update_user_age(new_id, 26)
user = get_user_by_id(new_id)
print(f"   수정 후 나이: {user['age']}\n")

# 5. 삭제 (Delete)
print(f"5. ID={new_id} 사용자 삭제")
success = delete_user(new_id)
print(f"   삭제 {'완료' if success else '실패'}\n")

# 최종 목록 확인
print("6. 최종 사용자 목록")
users = get_all_users()
for user in users:
    print(f"   [{user['id']}] {user['name']}")
```

실행 결과 예시입니다.

```
=== 사용자 관리 CRUD 실습 ===

1. 새 사용자 생성
   생성된 ID: 7

2. 전체 사용자 조회
   [1] 홍길동 - 29세
   [2] 김철수 - 35세
   [3] 이영희 - 24세
   [4] 박민준 - 42세
   [5] 최수진 - 31세
   [7] 테스트유저 - 25세

3. ID=7 사용자 조회
   이름: 테스트유저, 이메일: test@example.com

4. ID=7 사용자 나이 수정 (25 → 26)
   수정 후 나이: 26

5. ID=7 사용자 삭제
   삭제 완료

6. 최종 사용자 목록
   [1] 홍길동
   [2] 김철수
   [3] 이영희
   [4] 박민준
   [5] 최수진
```

> 📌 ***SQL 인젝션 방지*** *위 코드처럼 SQL 쿼리에 파라미터를 직접 문자열로 삽입하지 않고* `%s` *플레이스홀더와 튜플을 사용하면* ***SQL 인젝션(SQL Injection)*** *공격을 방어할 수 있습니다. 절대로* `f"... WHERE id = {user_id}"` *와 같이 f-string으로 직접 값을 삽입하지 마세요.*

마치며

이번 챕터에서는 데이터베이스의 기초부터 Python과의 연동까지 전체적인 흐름을 학습했습니다.

핵심 요약:

- **데이터베이스**는 관계형(RDBMS)과 NoSQL로 나뉘며, MySQL은 대표적인 관계형 DB입니다.
- **테이블**은 행(row)과 열(column)로 구성되며, **기본 키(PK)**로 각 행을 고유하게 식별합니다.
- SQL의 핵심 4명령어는 **SELECT**(조회), **INSERT**(삽입), **UPDATE**(수정), **DELETE**(삭제)입니다.
- **WHERE**, **ORDER BY**, **LIMIT**으로 데이터를 필터링, 정렬, 제한할 수 있습니다.
- **JOIN**으로 여러 테이블을 연결하고, **집계 함수**와 **GROUP BY**로 데이터를 요약합니다.
- Python에서는 **pymysql**로 직접 SQL을 실행하거나 **SQLAlchemy**로 ORM 방식으로 데이터베이스를 다룰 수 있습니다.
- SQL 파라미터는 반드시 플레이스홀더(`%s`)를 사용하여 **SQL 인젝션**을 방어해야 합니다.

다음 챕터 예고:

Chapter 20에서는 **RAG(Retrieval-Augmented Generation)**를 학습합니다. 이번 챕터에서 배운 데이터베이스 개념은 RAG 시스템에서 벡터 데이터베이스와 함께 활용됩니다. 데이터를 검색하고 LLM에 컨텍스트로 제공하는 과정에서 데이터베이스 기술이 핵심 역할을 합니다.

이번 챕터에서는 **RAG(Retrieval-Augmented Generation)**를 다룹니다. RAG는 LLM의 고질적인 한계인 지식 최신성 문제와 환각 현상을 외부 문서 검색으로 보완하는 기법입니다. 문서 수집부터 임베딩, 벡터 저장, 검색, 생성까지 전체 파이프라인을 단계별로 살펴보고, 마지막에는 ChromaDB와 GPT를 활용한 사내 문서 QA 시스템을 직접 구축합니다.

20.1 RAG란 무엇인가 — LLM의 한계를 보완하는 방법

대규모 언어 모델(**LLM**, Large Language Model)은 방대한 텍스트 데이터를 학습하여 자연스러운 문장을 생성하는 능력을 갖추고 있습니다. 그러나 LLM은 태생적으로 몇 가지 근본적인 한계를 지닙니다.

> *LLM은 학습 시점 이후의 정보를 알지 못하고, 존재하지 않는 사실을 그럴듯하게 만들어내는 **환각(Hallucination)** 현상이 발생할 수 있습니다.*

RAG(Retrieval-Augmented Generation)는 이러한 한계를 보완하기 위해 등장한 기법입니다. 질문에 답하기 전에 외부 지식 베이스에서 관련 문서를 **검색(Retrieval)**한 뒤, 검색된 내용을 컨텍스트로 삼아 LLM이 **생성(Generation)**하도록 합니다.

RAG가 해결하는 주요 문제는 다음과 같습니다.

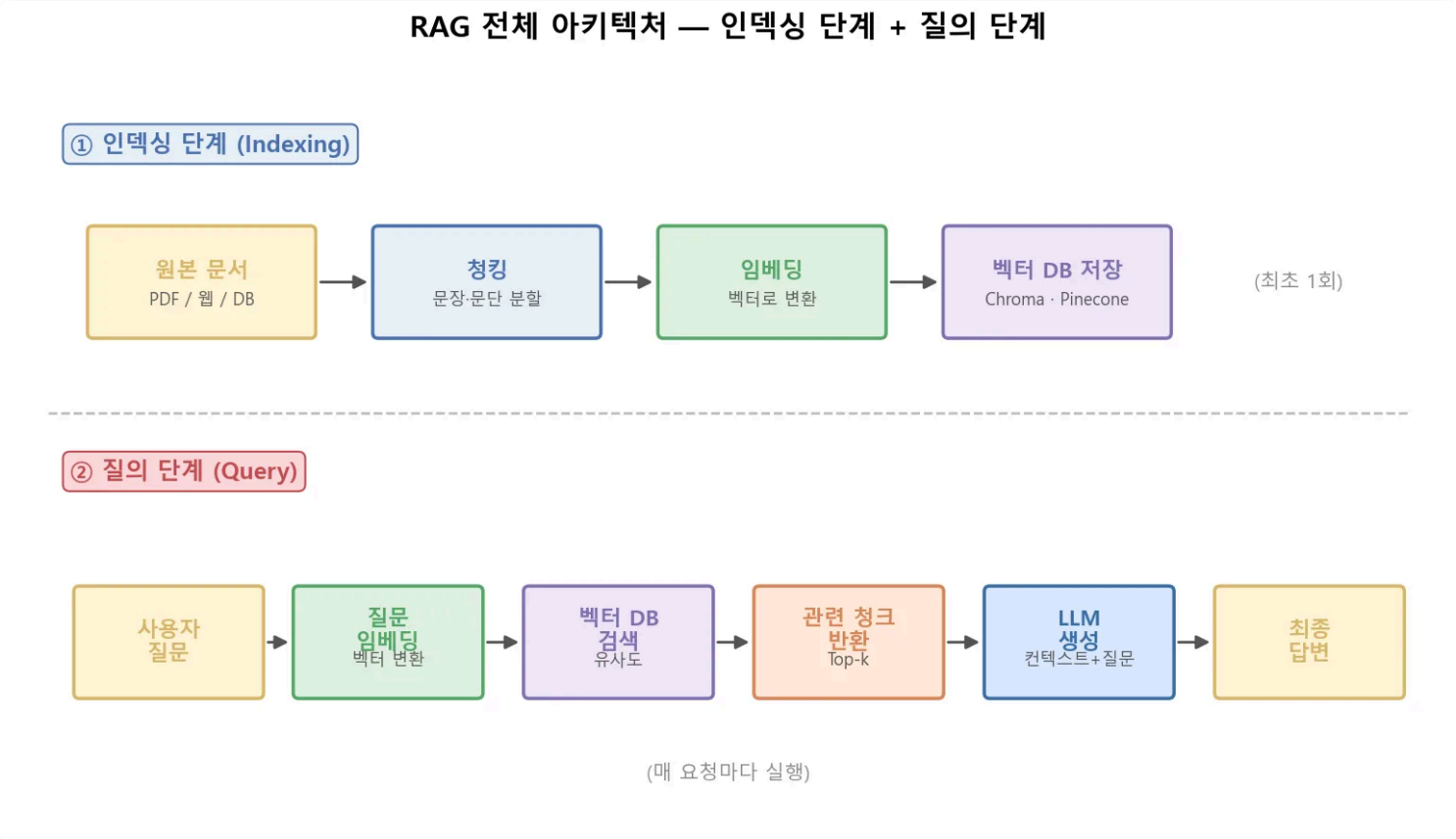
- **지식 최신성 문제**: 학습 데이터 이후 발생한 최신 정보를 반영할 수 있습니다.
- **환각(Hallucination) 문제**: 실제 문서를 근거로 답변하므로 허구의 정보 생성이 줄어듭니다.
- **도메인 특화 문제**: 사내 문서, 법률 문서 등 비공개 전문 지식을 활용할 수 있습니다.
- **비용 문제**: 모델 전체를 재학습(Fine-tuning)하지 않고도 지식을 갱신할 수 있습니다.

RAG와 Fine-tuning의 차이를 간단히 비교하면 아래와 같습니다.

구분	RAG	Fine-tuning
지식 갱신 방법	문서 DB 교체	모델 재학습
비용	낮음	높음
최신 정보 반영	즉시 가능	재학습 필요
근거 제시	검색 문서로 가능	어려움
적합한 상황	자주 바뀌는 지식	언어 스타일·태스크 특화

20.2 RAG의 전체 아키텍처

RAG 시스템은 크게 **인덱싱(Indexing)** 단계와 **질의(Query)** 단계로 나뉩니다.



각 단계를 순서대로 정리하면 다음과 같습니다.

1. **문서 수집(Document Loading)**: PDF, Word, 웹 페이지, 데이터베이스 등 다양한 소스에서 텍스트를 추출합니다.

2. **청킹(Chunking)**: 긴 문서를 LLM이 처리하기 적합한 크기의 조각으로 분할합니다.
3. **임베딩(Embedding)**: 각 청크를 수치 벡터로 변환합니다.
4. **벡터 DB 저장(Indexing)**: 변환된 벡터를 빠르게 검색할 수 있는 벡터 데이터베이스에 저장합니다.
5. **검색(Retrieval)**: 사용자 질문을 임베딩하고, 가장 유사한 청크를 벡터 DB에서 찾아옵니다.
6. **생성(Generation)**: 검색된 청크를 컨텍스트로 하여 LLM이 최종 답변을 생성합니다.

> *인덱싱은 최초 1회(또는 문서 갱신 시)만 수행하고, 질의 단계는 매 요청마다 실행됩니다.*

20.3 임베딩(Embedding)의 개념

임베딩(Embedding)이란 텍스트(단어, 문장, 문서)를 의미를 담은 고차원 숫자 벡터로 변환하는 과정입니다. 의미가 비슷한 문장은 벡터 공간에서 가까운 위치에 놓이게 됩니다.

OpenAI Embeddings API 사용

OpenAI는 `text-embedding-3-small`, `text-embedding-3-large` 등의 임베딩 모델을 제공합니다.

```
from openai import OpenAI

client = OpenAI()  # OPENAI_API_KEY 환경변수 사용

def get_embedding(text: str, model: str = "text-embedding-3-small") → list[float]:
    """텍스트를 임베딩 벡터로 변환합니다."""
    response = client.embeddings.create(
        input=text,
        model=model
    )
    return response.data[0].embedding

# 사용 예시
vector = get_embedding("인공지능이란 무엇인가?")
print(f"벡터 차원: {len(vector)}")  # text-embedding-3-small: 1536차원
```

문장 유사도 계산 — 코사인 유사도

두 벡터 간 유사도를 측정할 때는 **코사인 유사도(Cosine Similarity)**를 주로 사용합니다. 값이 1에 가까울수록 두 문장의 의미가 유사합니다.

```
import numpy as np

def cosine_similarity(vec_a: list[float], vec_b: list[float]) → float:
    """두 벡터의 코사인 유사도를 계산합니다."""
    a = np.array(vec_a)
    b = np.array(vec_b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

# 유사도 비교
sent1 = "강아지는 귀여운 동물입니다."
sent2 = "개는 사람에게 친근한 반려동물입니다."
sent3 = "오늘 날씨가 매우 맑습니다."

vec1 = get_embedding(sent1)
vec2 = get_embedding(sent2)
vec3 = get_embedding(sent3)

print(f"문장1 vs 문장2: {cosine_similarity(vec1, vec2):.4f}")  # 높은 유사도
print(f"문장1 vs 문장3: {cosine_similarity(vec1, vec3):.4f}")  # 낮은 유사도
```

> *`text-embedding-3-small` 은 비용 대비 성능이 우수하여 실습에 많이 사용합니다. 더 높은 정확도가 필요하면 `text-embedding-3-large` 를 선택합니다.*

20.4 벡터 데이터베이스

벡터 데이터베이스(Vector Database)는 고차원 벡터를 효율적으로 저장하고, 주어진 쿼리 벡터와 가장 가까운 벡터를 빠르게 검색(**ANN**, Approximate Nearest Neighbor)하도록 최적화된 데이터베이스입니다.

이름	특징	적합한 상황
ChromaDB	로컬 파일 또는 인메모리, 설치 간단	프로토타입·소규모 프로젝트
FAISS	Meta 개발, 고성능 로컬 라이브러리	대용량 오프라인 검색
Pinecone	완전 관리형 클라우드 서비스	프로덕션·확장성 필요 시
Weaviate	오픈소스, 하이브리드 검색 지원	복잡한 필터링·그래프 검색
Qdrant	Rust 기반, 빠른 속도	고성능 프로덕션 환경

ChromaDB 기본 사용법

```
import chromadb

# 인메모리 클라이언트 생성 (테스트용)
client = chromadb.Client()

# 영구 저장이 필요하면 아래 사용
# client = chromadb.PersistentClient(path="./chroma_db")

# 컬렉션 생성
collection = client.create_collection(name="my_documents")

# 문서 추가
collection.add(
    documents=["파이썬은 배우기 쉬운 언어입니다.", "RAG는 LLM의 한계를 보완합니다."],
    ids=["doc1", "doc2"]
)

# 유사 문서 검색
results = collection.query(
    query_texts=["파이썬 프로그래밍"],
    n_results=1
)
print(results["documents"])
```

FAISS 기본 사용법

```
import faiss
import numpy as np

# 1536차원 벡터용 인덱스 생성 (L2 거리)
dimension = 1536
index = faiss.IndexFlatL2(dimension)

# 벡터 추가 (numpy float32 배열 필요)
vectors = np.random.rand(100, dimension).astype("float32")
index.add(vectors)

# 가장 가까운 벡터 5개 검색
query = np.random.rand(1, dimension).astype("float32")
distances, indices = index.search(query, k=5)
print(f"검색된 인덱스: {indices[0]}")
```

20.5 문서 청킹(Chunking) 전략

청킹(Chunking)은 긴 문서를 LLM 컨텍스트 윈도우에 맞는 작은 단위로 분할하는 과정입니다. 청킹 전략은 검색 품질에 직접적인 영향을 미칩니다.

고정 크기 청킹 (Fixed-size Chunking)

문자 수 또는 토큰 수를 기준으로 일정 크기로 분할합니다. 구현이 단순하지만 문장이나 문단이 중간에 잘릴 수 있습니다.

```
def fixed_size_chunk(text: str, chunk_size: int = 500, overlap: int = 50) → list[str]:
    """고정 크기로 텍스트를 청킹합니다. overlap으로 문맥 연속성을 유지합니다."""
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap  # overlap만큼 겹치게 이동
    return chunks

text = "긴 문서 내용..." * 100
chunks = fixed_size_chunk(text, chunk_size=200, overlap=30)
print(f"총 청크 수: {len(chunks)}")
```

문장 기반 청킹 (Sentence-based Chunking)

문장 단위로 분할하여 의미 단위가 잘리지 않도록 합니다.

```
import re

def sentence_chunk(text: str, max_chunk_size: int = 500) → list[str]:
    """문장 단위로 청킹합니다."""
    # 문장 분리 (마침표, 느낌표, 물음표 기준)
    sentences = re.split(r"(?⇐[.!?])\s+", text)
    chunks = []
    current_chunk = ""

    for sentence in sentences:
        # 현재 청크에 문장을 추가했을 때 최대 크기를 초과하면 청크 저장
        if len(current_chunk) + len(sentence) > max_chunk_size and current_chunk:
            chunks.append(current_chunk.strip())
            current_chunk = sentence
        else:
            current_chunk += " " + sentence

    if current_chunk.strip():
        chunks.append(current_chunk.strip())

    return chunks
```

의미 기반 청킹 (Semantic Chunking)

임베딩 유사도를 기반으로 의미가 바뀌는 지점을 감지하여 분할합니다. 품질이 가장 높지만 처리 비용이 높습니다.

```
def semantic_chunk(sentences: list[str], threshold: float = 0.7) → list[str]:
    """인접한 문장의 임베딩 유사도가 threshold 미만이면 새 청크를 시작합니다."""
    if not sentences:
        return []

    chunks = []
    current_chunk = [sentences[0]]
    prev_vec = get_embedding(sentences[0])

    for sentence in sentences[1:]:
        curr_vec = get_embedding(sentence)
        similarity = cosine_similarity(prev_vec, curr_vec)

        if similarity < threshold:
            # 의미가 크게 바뀌는 지점 → 새 청크 시작
            chunks.append(" ".join(current_chunk))
            current_chunk = [sentence]
        else:
            current_chunk.append(sentence)

        prev_vec = curr_vec

    if current_chunk:
        chunks.append(" ".join(current_chunk))

    return chunks
```

세 가지 전략을 비교하면 아래와 같습니다.

전략	구현 난이도	의미 보존	처리 비용	권장 상황
고정 크기	낮음	낮음	낮음	빠른 프로토타입
문장 기반	중간	중간	낮음	일반적인 문서
의미 기반	높음	높음	높음	고품질 검색 필요 시

> *실무에서는 **문장 기반 + 일정 overlap** 조합이 가장 무난한 출발점입니다.*

20.6 검색(Retrieval)과 생성(Generation) 파이프라인 연결

이제 검색과 생성을 연결하는 **RAG 파이프라인**을 구성합니다.

기본 RAG 파이프라인 구조

```
from openai import OpenAI
import chromadb

client = OpenAI()
chroma_client = chromadb.PersistentClient(path="./chroma_db")
collection = chroma_client.get_or_create_collection(name="documents")

def retrieve(query: str, n_results: int = 3) → list[str]:
    """쿼리와 가장 유사한 문서 청크를 검색합니다.

    중요: 인덱싱 시 OpenAI 임베딩을 명시적으로 넣었으므로, 검색 시에도
    동일한 임베딩 함수로 만든 벡터를 `query_embeddings`로 넘겨야 합니다.
    `query_texts`를 쓰면 ChromaDB의 기본 임베더로 쿼리가 인코딩되어
    저장된 벡터와 공간이 달라져 검색 정확도가 크게 떨어집니다.
    """
    query_embedding = get_embedding(query)
    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=n_results
    )
    return results["documents"][0]  # 가장 유사한 n개 청크 반환

def generate(query: str, context_chunks: list[str]) → str:
    """검색된 컨텍스트를 바탕으로 LLM이 답변을 생성합니다."""
    context = "\n\n".join(context_chunks)

    system_prompt = """당신은 주어진 문서를 바탕으로 질문에 답하는 도우미입니다.
반드시 제공된 문서 내용을 근거로 답변하고, 문서에 없는 내용은 '문서에서 찾을 수 없습니다'라고 답하세
요."""

    user_prompt = f"""[참고 문서]
{context}

[질문]
{query}"""

    response = client.chat.completions.create(
        model="gpt-5-mini",           # 2026년 기준 저비용 모델. 폐기 시 동급 최신 모델로 교체
        messages=[
            {"role": "system", "content": system_prompt},
```

```
            {"role": "user", "content": user_prompt}
        ]
    )
    return response.choices[0].message.content

def rag_pipeline(query: str) → str:
    """RAG 파이프라인: 검색 후 생성합니다."""
    # 1. 검색
    context_chunks = retrieve(query, n_results=3)
    # 2. 생성
    answer = generate(query, context_chunks)
    return answer

# 실행
answer = rag_pipeline("파이썬에서 리스트 컴프리헨션은 어떻게 사용하나요?")
print(answer)
```

문서 인덱싱 함수

```
def index_documents(documents: list[str], ids: list[str] | None = None) → None:
    """문서 리스트를 청킹 후 임베딩하여 벡터 DB에 저장합니다."""
    all_chunks = []
    all_ids = []

    for i, doc in enumerate(documents):
        chunks = sentence_chunk(doc, max_chunk_size=500)
        for j, chunk in enumerate(chunks):
            all_chunks.append(chunk)
            all_ids.append(f"doc{i}_chunk{j}")

    # ChromaDB는 내부적으로 임베딩을 자동 처리할 수도 있지만,
    # 명시적으로 OpenAI 임베딩을 사용하려면 아래와 같이 합니다.
    embeddings = [get_embedding(chunk) for chunk in all_chunks]

    collection.add(
        documents=all_chunks,
        embeddings=embeddings,
        ids=all_ids
    )
    print(f"총 {len(all_chunks)}개 청크가 인덱싱되었습니다.")
```

20.7 RAG 성능 최적화

기본 RAG 파이프라인을 구축한 후에는 검색 품질과 생성 품질을 높이는 최적화 작업이 필요합니다.

리랭킹(Re-ranking)

리랭킹(Re-ranking)은 1차 검색으로 후보 문서를 넉넉하게 가져온 뒤(예: 상위 10개), 더 정교한 모델로 질문과의 관련도를 재평가하여 최종 상위 k개만 사용하는 기법입니다.

```
from openai import OpenAI

client = OpenAI()

def rerank_with_llm(query: str, chunks: list[str], top_k: int = 3) → list[str]:
    """LLM을 사용하여 청크의 관련도를 재평가하고 상위 top_k개를 반환합니다."""
    scored_chunks = []

    for chunk in chunks:
        prompt = f"""다음 문서가 질문에 얼마나 관련이 있는지 0~10 사이의 숫자만 출력하세요.

질문: {query}
문서: {chunk}

점수 (숫자만):"""

        response = client.chat.completions.create(
            model="gpt-5-mini",        # 폐기 시 동급 최신 모델로 교체
            messages=[{"role": "user", "content": prompt}],
            max_tokens=5
        )
        try:
            score = float(response.choices[0].message.content.strip())
        except ValueError:
            score = 0.0
        scored_chunks.append((score, chunk))

    # 점수 내림차순 정렬 후 top_k 반환
    scored_chunks.sort(key=lambda x: x[0], reverse=True)
    return [chunk for _, chunk in scored_chunks[:top_k]]
```

하이브리드 검색 (Hybrid Search)

하이브리드 검색(Hybrid Search)은 벡터 유사도 검색(**Dense Retrieval**)과 키워드 기반 검색(**Sparse Retrieval**, BM25)을 결합하여 두 방식의 장점을 취합니다.

```
from rank_bm25 import BM25Okapi  # pip install rank-bm25

class HybridRetriever:
    """BM25(키워드)와 벡터 검색을 결합한 하이브리드 검색기입니다."""

    def __init__(self, chunks: list[str], alpha: float = 0.5):
        self.chunks = chunks
        self.alpha = alpha  # 벡터 검색 가중치 (1-alpha: BM25 가중치)

        # BM25 인덱스 구축
        tokenized = [chunk.split() for chunk in chunks]
        self.bm25 = BM25Okapi(tokenized)

        # 벡터 인덱스 구축
        self.embeddings = [get_embedding(chunk) for chunk in chunks]

    def search(self, query: str, top_k: int = 5) → list[str]:
        # BM25 점수
        bm25_scores = self.bm25.get_scores(query.split())

        # 벡터 유사도 점수
        query_vec = get_embedding(query)
        vector_scores = [cosine_similarity(query_vec, emb) for emb in self.embeddings]

        # 정규화 후 결합
        bm25_norm = bm25_scores / (bm25_scores.max() + 1e-9)
        vector_norm = [s / (max(vector_scores) + 1e-9) for s in vector_scores]

        combined = [
            self.alpha * v + (1 - self.alpha) * b
            for v, b in zip(vector_norm, bm25_norm)
        ]

        # 상위 top_k 인덱스 추출
        top_indices = sorted(range(len(combined)), key=lambda i: combined[i], reverse=True)[:t
op_k]
        return [self.chunks[i] for i in top_indices]
```

프롬프트 최적화

검색된 컨텍스트를 LLM에게 전달하는 방식도 성능에 큰 영향을 미칩니다.

```
def build_optimized_prompt(query: str, chunks: list[str]) → str:
    """출처 정보를 포함한 최적화 프롬프트를 구성합니다."""
    context_parts = []
    for i, chunk in enumerate(chunks, 1):
        context_parts.append(f"[문서 {i}]\n{chunk}")
    context = "\n\n".join(context_parts)

    return f"""다음 문서들을 참고하여 질문에 답변하세요.
답변 시 반드시 어느 문서를 근거로 했는지 [문서 N] 형식으로 출처를 밝히세요.
문서에 없는 내용은 추측하지 말고 '해당 정보를 찾을 수 없습니다'라고 답하세요.

{context}

질문: {query}
답변:"""
```

주요 최적화 전략을 정리하면 아래와 같습니다.

최적화 기법	효과	추가 비용
리랭킹(Re-ranking)	검색 정확도 향상	LLM 추가 호출
하이브리드 검색	키워드 누락 방지	BM25 인덱스 구축
청크 크기 튜닝	컨텍스트 품질 향상	재인덱싱 필요
프롬프트 최적화	답변 품질·출처 명시	없음
메타데이터 필터링	관련 없는 문서 제거	메타데이터 관리

[실습] 사내 문서 기반 QA 시스템 구축 (ChromaDB + GPT)

지금까지 배운 내용을 종합하여 **사내 문서 QA 시스템**을 구축합니다. 텍스트 파일 형태의 사내 문서를 읽어 ChromaDB에 인덱싱하고, 자연어 질문으로 답변을 받는 시스템을 만들겠습니다.

패키지 설치

```
pip install openai chromadb tiktoken
```

전체 코드

```
# qa_system.py
import os
import re
from pathlib import Path
from openai import OpenAI
import chromadb

# ── 초기화 ──────────────────────────────────────────
client = OpenAI()  # OPENAI_API_KEY 환경변수 필요
chroma_client = chromadb.PersistentClient(path="./chroma_db")
collection = chroma_client.get_or_create_collection(
    name="company_docs",
    metadata={"hnsw:space": "cosine"}  # 코사인 거리 사용
)

# ── 임베딩 ──────────────────────────────────────────
def get_embedding(text: str) → list[float]:
    """OpenAI API로 텍스트를 임베딩 벡터로 변환합니다."""
    response = client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding

# ── 청킹 ──────────────────────────────────────────
def chunk_text(text: str, chunk_size: int = 400, overlap: int = 50) → list[str]:
    """문장 단위를 고려한 고정 크기 청킹을 수행합니다."""
    sentences = re.split(r"(?⇐[.!?\n])\s*", text)
    chunks, current = [], ""
    for sentence in sentences:
        if len(current) + len(sentence) > chunk_size and current:
            chunks.append(current.strip())
            # overlap: 마지막 문장을 다음 청크에 포함
            current = current[-overlap:] + sentence
        else:
            current += sentence
    if current.strip():
        chunks.append(current.strip())
    return [c for c in chunks if len(c) > 20]  # 너무 짧은 청크 제거
```

```
# ── 인덱싱 ──────────────────────────────────────────
def index_file(file_path: str) → int:
    """파일을 읽어 청킹 후 ChromaDB에 저장합니다."""
    path = Path(file_path)
    text = path.read_text(encoding="utf-8")
    chunks = chunk_text(text)

    # 이미 인덱싱된 문서 확인 (중복 방지)
    file_name = path.stem
    existing = collection.get(where={"source": file_name})
    if existing["ids"]:
        print(f"[건너뜀] '{file_name}'은 이미 인덱싱되어 있습니다.")
        return 0

    embeddings = [get_embedding(chunk) for chunk in chunks]
    ids = [f"{file_name}_chunk{i}" for i in range(len(chunks))]
    metadatas = [{"source": file_name, "chunk_index": i} for i in range(len(chunks))]

    collection.add(
        documents=chunks,
        embeddings=embeddings,
        ids=ids,
        metadatas=metadatas
    )
    print(f"[완료] '{file_name}' → {len(chunks)}개 청크 인덱싱")
    return len(chunks)

def index_directory(dir_path: str) → None:
    """디렉토리 내 모든 .txt 파일을 인덱싱합니다."""
    total = 0
    for file in Path(dir_path).glob("*.txt"):
        total += index_file(str(file))
    print(f"\n총 {total}개 청크가 벡터 DB에 저장되었습니다.")

# ── 검색 ──────────────────────────────────────────
def retrieve(query: str, n_results: int = 3) → list[dict]:
    """질문과 가장 유사한 청크를 반환합니다."""
    query_embedding = get_embedding(query)
    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=n_results,
        include=["documents", "metadatas", "distances"]
    )
```

```
    retrieved = []
    for doc, meta, dist in zip(
        results["documents"][0],
        results["metadatas"][0],
        results["distances"][0]
    ):
        retrieved.append({
            "content": doc,
            "source": meta["source"],
            "similarity": 1 - dist  # 코사인 거리 → 유사도로 변환
        })
    return retrieved

# ── 생성 ──────────────────────────────────────────
def generate_answer(query: str, retrieved_chunks: list[dict]) → str:
    """검색된 청크를 컨텍스트로 GPT가 답변을 생성합니다."""
    context_parts = []
    for i, chunk in enumerate(retrieved_chunks, 1):
        context_parts.append(
            f"[문서 {i} | 출처: {chunk['source']} | 유사도: {chunk['similarity']:.2f}]\n{chunk
['content']}"
        )
    context = "\n\n".join(context_parts)

    system_msg = """당신은 회사 내부 문서를 기반으로 직원들의 질문에 답하는 AI 도우미입니다.
- 반드시 제공된 문서 내용만을 근거로 답변하세요.
- 답변 마지막에 참고한 출처 문서명을 명시하세요.
- 문서에서 찾을 수 없는 내용은 "해당 정보를 문서에서 찾을 수 없습니다"라고 답하세요."""

    user_msg = f"""참고 문서:
{context}

질문: {query}"""

    response = client.chat.completions.create(
        model="gpt-5-mini",            # 2026년 기준 저비용 모델. 폐기 시 동급 최신 모델로 교체
        messages=[
            {"role": "system", "content": system_msg},
            {"role": "user", "content": user_msg}
        ],
        temperature=0.1  # 사실 기반 답변을 위해 낮은 temperature 사용
    )
    return response.choices[0].message.content
```

```python
# ── QA 파이프라인 ──────────────────────────────────────
def ask(question: str) → str:
    """질문을 받아 RAG 파이프라인을 실행하고 답변을 반환합니다."""
    print(f"\n질문: {question}")
    print("-" * 50)

    # 검색
    chunks = retrieve(question, n_results=3)
    print(f"검색된 청크 수: {len(chunks)}")
    for c in chunks:
        print(f"  - [{c['source']}] 유사도: {c['similarity']:.3f}")

    # 생성
    answer = generate_answer(question, chunks)
    print(f"\n답변:\n{answer}")
    return answer

# ── 실행 예시 ──────────────────────────────────────────
if __name__ == "__main__":
    # 1. 문서 인덱싱 (최초 1회)
    index_directory("./company_docs")

    # 2. 질의응답
    ask("연차 휴가는 몇 일 주어지나요?")
    ask("재택근무 신청 방법을 알려주세요.")
    ask("경비 처리 절차는 어떻게 되나요?")
```

테스트용 샘플 문서 생성

```
# create_sample_docs.py — 테스트용 사내 문서 생성 스크립트
import os

os.makedirs("company_docs", exist_ok=True)

docs = {
    "인사규정.txt": """
연차 휴가 규정

1. 연차 휴가 일수
신입사원은 입사 첫 해에 11일의 연차 휴가가 주어집니다.
만 1년 이상 근무한 직원은 15일의 연차 휴가를 받습니다.
매 2년마다 1일씩 추가되며 최대 25일까지 부여됩니다.

2. 연차 신청 방법
연차 휴가는 사내 인사 포털(HR Portal)에서 신청합니다.
최소 3일 전에 신청해야 하며, 팀장 승인 후 확정됩니다.
긴급한 경우 당일 신청도 가능하나 팀장과 사전 협의가 필요합니다.

3. 연차 소진 기준
미사용 연차는 해당 연도 12월 31일까지 소진해야 합니다.
미소진 연차는 다음 연도로 이월되지 않으며 자동 소멸됩니다.
""",
    "재택근무지침.txt": """
재택근무 운영 지침

1. 재택근무 대상
전 직원을 대상으로 주 2일 재택근무가 허용됩니다.
단, 신입사원은 입사 후 3개월간 재택근무가 제한됩니다.

2. 신청 방법
재택근무는 매주 금요일까지 다음 주 일정을 사내 포털에 등록합니다.
팀장 승인 없이도 신청 가능하나, 팀 회의 일정과 겹치지 않아야 합니다.

3. 재택근무 시 준수 사항
업무 시간(09:00~18:00)에는 메신저 및 이메일을 즉시 응답해야 합니다.
화상 회의 참여 시 카메라를 켜는 것을 원칙으로 합니다.
사내 VPN에 접속하여 업무를 수행해야 합니다.
""",
    "경비처리규정.txt": """
```

```
경비 처리 규정

1. 경비 처리 가능 항목
업무 관련 교통비, 식대, 숙박비, 소모품 구입비가 경비 처리 대상입니다.
식대는 1인당 1만 5천원을 한도로 합니다.
택시비는 대중교통 운행 종료 후(자정 이후) 또는 긴급 업무 시에만 인정됩니다.

2. 경비 신청 방법
영수증을 스캔하여 경비 처리 시스템에 업로드합니다.
신청 마감은 매월 25일이며, 익월 급여일에 함께 지급됩니다.
50만원 이상의 경비는 팀장 및 재무팀의 사전 승인이 필요합니다.

3. 법인카드 사용
법인카드 사용 후 3일 이내에 용도와 참석자를 시스템에 등록해야 합니다.
개인적인 용도로 법인카드를 사용하는 것은 엄격히 금지됩니다.
"""
}

for filename, content in docs.items():
    with open(f"company_docs/{filename}", "w", encoding="utf-8") as f:
        f.write(content.strip())

print("샘플 문서 생성 완료!")
```

실행 결과 예시

```
샘플 문서 생성 완료!
[완료] '인사규정' → 4개 청크 인덱싱
[완료] '재택근무지침' → 3개 청크 인덱싱
[완료] '경비처리규정' → 4개 청크 인덱싱

총 11개 청크가 벡터 DB에 저장되었습니다.

질문: 연차 휴가는 몇 일 주어지나요?
--------------------------------------------------
검색된 청크 수: 3
  - [인사규정] 유사도: 0.921
  - [인사규정] 유사도: 0.843
  - [재택근무지침] 유사도: 0.412

답변:
신입사원은 입사 첫 해에 11일, 만 1년 이상 근무한 직원은 15일의 연차 휴가가 주어집니다.
이후 매 2년마다 1일씩 추가되어 최대 25일까지 부여됩니다.

[출처: 인사규정]
```

> *실제 서비스에서는 PDF 파싱(`pdfplumber` , `pymupdf`), 웹 크롤링 결과, 데이터베이스 덤프 등 다양한 소스의 문서를 인덱싱할 수 있습니다. LangChain이나 LlamaIndex 같은 프레임워크를 사용하면 이 과정을 더욱 편리하게 구성할 수 있습니다.*

정리

이번 챕터에서는 RAG의 전체 흐름을 학습하고 실습했습니다.

- **RAG**는 LLM의 지식 최신성·환각 문제를 외부 문서 검색으로 보완하는 기법입니다.
- 전체 파이프라인은 **문서 수집 → 청킹 → 임베딩 → 벡터 DB 저장 → 검색 → 생성** 순서로 동작합니다.
- **임베딩**은 텍스트를 의미 벡터로 변환하며, 코사인 유사도로 문장 간 유사도를 측정합니다.
- **벡터 데이터베이스**는 ChromaDB(로컬), FAISS(고성능), Pinecone(클라우드) 등이 있습니다.
- **청킹 전략**은 고정 크기, 문장 기반, 의미 기반으로 나뉘며 검색 품질에 큰 영향을 줍니다.
- **리랭킹, 하이브리드 검색, 프롬프트 최적화**로 RAG 성능을 높일 수 있습니다.

다음 챕터에서는 Part 8의 첫 번째 주제인 **웹 애플리케이션 기초**를 다룹니다. Flask와 FastAPI를 활용하여 파이썬으로 웹 서버를 구축하는 방법을 학습합니다.